

Final Report

Discrete Diffusion for Anatomical Shape Generation in the Prostate Domain

Miguel Angel Rodriguez Fuentes

Télécom Paris – Institut Polytechnique de Paris

Academic year 2025–2026

The project repository: PRIM-Discrete-Difusion-for-Anatomical-Shape-Generation

Abstract

This report presents a compact pipeline for generating 3D anatomical shapes in the prostate domain [1, 2, 3, 4]. The work follows the general PRIM concept of learning anatomy in discrete latent space, but the project’s original focus on the heart was adapted here to prostate segmentation masks. For the tokenizer stage, the initial VQ-VAE implementation was taken from the public GitHub repository of AinkaranSanthi and then adapted to the prostate setting and to the final PRIM pipeline [5]. The process consists of five stages: balanced split generation, VQ-VAE training, token export and analysis, discrete diffusion training on token grids, and decoding the generated tokens into complete 3D masks.

The VQ-VAE tokenizer was trained for 50 epochs. The best control point was obtained at epoch 49 for both monitored criteria. The Dice validation coefficient reached 0.7975, and the multiple validation score reached 0.6812. In the 12-case test set, the tokenizer achieved a mean Dice coefficient of 0.8025, a mean Hausdorff distance of 11.9389, and a mean surface distance of 1.6778. Token analysis showed that only 10 of the 32 codebook entries were active in the real data, indicating a stable but concentrated latent representation.

For the discrete diffusion stage, nine a priori token configurations were compared: a reference experiment and eight variants. Experiment 0 yielded the best validation loss (0.1852), while Experiment 1.1 offered the best similarity metrics for the decoded samples. The decoded samples are anatomically plausible and preserve the overall structure of the prostate, but the complete set still exhibits moderate diversity. Overall, the results support the use of discrete latent modeling for compact anatomical representation and plausible 3D shape generation.

1 Introduction

Learning realistic models of anatomy is useful in medical imaging because it can support segmentation, shape analysis, and data generation. Direct generation in voxel space is expensive, especially for 3D volumes. A common alternative is to first compress the object into a compact latent space and then learn a generative model in that space [2, 1]. This is the main idea of the PRIM project. In the present implementation, the tokenizer stage starts from the public VQ-VAE segmentation repository released by AinkaranSanthi, which was then adapted to the prostate task and integrated into the final pipeline [5].

The original PRIM description was written for cardiac anatomy, but the present work applies the same logic to prostate anatomy. This adaptation is consistent with the project goal of anatomical shape generation, while changing the application domain to prostate MRI segmentations [3, 4, 6]. In this report, the objects of interest are the transition zone (TZ) and the peripheral zone (PZ). The central question is simple: can a VQ-VAE learn a useful symbolic representation of prostate masks, and can a discrete diffusion prior generate plausible new token grids from that representation?

The final pipeline is organized in five stages. First, the dataset is split in a balanced way according to prostate-zone volumes. Second, a 3D VQ-VAE tokenizer learns a compact grid of discrete codes from segmentation masks. Third, the exported tokens are analyzed with slice plots and histograms. Fourth, a discrete diffusion prior is trained on token grids. Fifth, sampled token volumes are decoded back into full 3D segmentations. The following sections describe these stages and the corresponding results.

2 Overall Pipeline Methodology

The final implementation of this project follows a sequential five-stage pipeline, orchestrated from the notebook `PRIM_pipeline.ipynb`. Rather than treating tokenization and generation as isolated experiments, the workflow was designed as an end-to-end process in which each stage produces the inputs required by the next one. The complete pipeline is composed of: (i) balanced split generation, (ii) training of a 3D vector-quantized tokenizer, (iii) token export and token-space diagnostics, (iv) training of a discrete diffusion prior over token grids, and (v) decoding of generated tokens into full-resolution 3D segmentation masks. This design follows the general idea of first learning a compact discrete latent representation [2] and then modeling that latent space with discrete diffusion [1].

2.1 Pipeline overview.

The practical execution order of the project is defined by the following scripts:

1. `make_balanced_split_by_volume.py`
2. `trainprostVQ.py`
3. `tokens.py`
4. `train_token_prior.py`
5. `decode_samples.py`

All intermediate and final outputs are stored under `./data/Prostate/outputprostatefinal`. This directory contains trained checkpoints, exported token tensors, token visualizations, sampled token grids from the prior, and decoded volumetric segmentations.

2.2 Stage 0: Balanced split generation.

The pipeline begins with the generation of a new train/validation/test split using the script `make_balanced_split_by_volume.py`. This script reads the available case list, loads the segmentation masks, computes voxel counts and physical volumes for the transition zone (TZ) and peripheral zone (PZ), infers a patient identifier from the file paths, and builds a balanced split at the patient level. The balancing strategy is based on quantile-derived strata computed from TZ and PZ volumes, so that the resulting subsets are not only separated by patient, but also better distributed in terms of anatomical size. In the final experiment, this stage produced 55 training cases, 12 validation cases, and 12 test cases. In addition to the split CSV files, the script also exports a volume summary table used to document the anatomical distribution of the dataset.

2.3 Stage 1: Training of the VQ tokenizer.

Once the data partition is fixed, the next step is to train the tokenizer with `trainprostVQ.py`. This script implements the full training, validation, testing, checkpointing, and token export logic for the first stage of the pipeline. The tokenizer is based on the `VQUNet3Dposv3` architecture

implemented in `convnet3D_utils.py`, which combines a 3D U-Net backbone, residual blocks, positional encoding, and a vector-quantization bottleneck. This stage starts from the public VQ-VAE segmentation code by AinkaranSanthi and adapts it to the prostate setting used in this report [5]. In the present adaptation, the model is trained directly on segmentation masks rather than raw MR intensities, and integer label volumes are converted to one-hot tensors. An additional coarse boundary channel is optionally appended in order to emphasize anatomical interfaces.

For the final experiment, the tokenizer was trained in a three-class setting (background, TZ, and PZ) with `channels=16`, `embed_dim=256`, and a codebook size of `n_embed=32`. The model was trained for 50 epochs. During training, the script monitors not only the validation Dice score but also Hausdorff distance (HD), average surface distance (ASD), and a composite score denoted `val_multi`. Two best checkpoints are therefore maintained: one selected by Dice and one selected by the composite metric. In the experiment reported here, both selection criteria converged to the same checkpoint at epoch 49, indicating that the best reconstruction model was also the best according to the multi-metric criterion.

2.4 Stage 2: Token export and latent-space diagnostics.

After training the tokenizer, `trainprostVQ.py` exports token grids for the train, validation, and test subsets as `tokens_train.pt`, `tokens_val.pt`, and `tokens_test.pt`. These files contain the discrete latent representations produced by the encoder-quantizer stage of the VQ model. In this implementation, each 3D segmentation volume is compressed into a token grid of size $12 \times 12 \times 4$, which defines the latent spatial domain used by the generative model.

2.5 Stage 3: Training of the discrete diffusion prior.

The third modeling stage is implemented in `train_token_prior.py`, with the core diffusion utilities defined in `token_diffusion.py`. At this stage, the input is no longer a segmentation mask, but a discrete token tensor of shape $[N, H, W, D]$. The objective is to learn a prior over token grids that can reverse a corruption process and progressively generate plausible new latent anatomical configurations.

2.6 Stage 4: Decoding generated tokens into anatomical segmentations.

The last stage of the pipeline is performed by `decode_samples.py`. This script closes the generative loop by taking a generated token file from the prior and decoding it back into full-resolution 3D segmentation masks using the best available tokenizer checkpoint. Concretely, the script automatically retrieves the highest-epoch `best_dice` tokenizer checkpoint and the preferred generated token file, giving priority to `samples_tokens_best.pt`. The token tensor is then passed through the decoder part of the trained VQ model, and the resulting segmentations are exported as NIfTI volumes.

3 Data and Preprocessing

3.1 Dataset adaptation to the prostate setting.

Although the original PRIM project was initially formulated in the context of cardiac anatomy generation, the implementation developed in this work was adapted to the prostate domain. The experiments were conducted using volumetric prostate data derived from the *PROSTATE-DIAGNOSIS* and *Prostate-3T* collections [3, 4], which provide T2-weighted axial MRI scans together with expert annotations of the transition zone (TZ) and peripheral zone (PZ). In the present work, however, the final generative pipeline does not model raw MRI intensities directly.

Instead, it operates on the anatomical segmentation masks themselves, since the main objective is to learn and generate plausible 3D anatomical shapes rather than image appearance.

3.2 Nature of the model inputs.

A key adaptation introduced in this project is that the tokenizer stage is trained directly on segmentation masks. In practice, the data loading pipeline keeps MONAI dictionary conventions, but both the “image” and “label” entries point to the same segmentation mask. This design makes the objective explicit: the model is trained to encode and reconstruct anatomical label volumes, not grayscale MR images. The segmentation task is defined in a three-class setting composed of background, transition zone (TZ), and peripheral zone (PZ). During training, integer masks are converted to one-hot representations, and an additional boundary channel is appended in order to emphasize coarse anatomical interfaces between regions. This yields a structured multi-channel input specifically designed for anatomical tokenization.

3.3 Preprocessing pipeline.

Preprocessing was standardized across training, validation, and testing in order to ensure spatial consistency of the volumes used by the tokenizer. First, masks were loaded and converted to channel-first format. Then, all volumes were resampled to a common voxel spacing of (0.5, 0.5, 1.5) using nearest-neighbor interpolation, preserving the discrete label nature of the masks. A foreground crop based on the segmentation mask was subsequently applied, followed by a reorientation to the RAS coordinate system. All samples were then padded to a fixed spatial size of (192, 192, 64), which defines the spatial support used throughout the experiments.

For the training set, additional volumetric data augmentation was applied to improve robustness and reduce overfitting. This included positive-label-centered random cropping at the target spatial size, random flipping along one spatial axis, and random 3D rotations. Validation and test data were processed with the same geometric standardization steps.

3.4 Original split limitations.

The initial repository relied on a center-based split in which training and validation data came from BMC, while test data came exclusively from RUNMC. Such a division may introduce undesirable bias because scanner characteristics, acquisition settings, annotation style, or patient distributions may differ across centers. In addition, the original split did not explicitly control for anatomical balance with respect to the relative sizes of the TZ and PZ regions. Since the main purpose of this project is to learn a discrete representation of anatomical variability, an evaluation protocol strongly tied to acquisition center would be less informative than a split that better reflects shape variability across the full dataset.

3.5 Balanced split generation.

To obtain a more robust and anatomically balanced protocol, the original center-based split was replaced by a new split generated with the custom script `make_balanced_split_by_volume.py`. This script reads the available case list, loads the corresponding segmentation masks, computes voxel counts and physical volumes (in mm^3) for the TZ and PZ classes, and infers a patient identifier from the file paths. Based on these statistics, cases are grouped into volume-based strata using TZ and PZ quantiles, and the final partition is performed at the patient level in order to avoid placing volumes from the same patient in different subsets.

In the final experimental setup, this procedure produced a split of 55 training cases, 12 validation cases, and 12 test cases. As a result, the generated `train.csv`, `validation.csv`, and `test.csv` files contain a mixture of cases originating from both BMC and RUNMC, while remaining better balanced in terms of anatomical region size. This redesign reduces the risk

of center-specific evaluation bias and makes the subsequent tokenizer and diffusion experiments more representative of anatomical variability rather than dataset provenance.

4 Anatomical Tokenizer Based on a 3D VQ-UNet

The first modeling stage of the final pipeline consists of learning a discrete latent representation of prostate anatomy from segmentation masks. Rather than performing generation directly in voxel space, the approach first compresses each anatomical volume into a compact grid of discrete codes, following the general principle of vector-quantized latent modeling [2]. In the present implementation, this tokenizer stage is implemented in `trainprostVQ.py`, while its main architectural components are defined in `convnet3D_utils.py`. The resulting token grids constitute the latent representation later used by the discrete diffusion prior.

4.1 Tokenizer architecture

The core tokenizer model is `VQUNet3Dposv3`, a 3D VQ-UNet architecture composed of an encoder, a vector-quantization bottleneck, and a decoder. The encoder progressively reduces spatial resolution through a sequence of 3D convolutions and residual blocks, while increasing the number of feature channels. Specifically, the architecture starts from a first convolutional stage at `channels=16`, followed by four downsampling stages that expand the feature width up to `channels × 16`. At the bottleneck, a $1 \times 1 \times 1$ convolution maps the deepest features to the embedding dimension, which is then discretized through a vector-quantization module. The decoder mirrors this process with a post-quantization projection, residual blocks, and trilinear upsampling layers followed by 3D convolutions, finally producing voxelwise segmentation logits through a last $1 \times 1 \times 1$ convolution.

Before being passed to the network, the integer-valued labels are transformed by the `build_x` function. First, the three-class segmentation mask (background, TZ, PZ) is converted into a one-hot tensor. Then, because `use_boundary_channel=True` in the final experiment, a coarse boundary map is computed from the one-hot labels by morphological dilation and erosion and concatenated as an additional input channel. As a result, the effective input to the tokenizer has four channels: three semantic one-hot channels plus one boundary channel. This boundary augmentation is important because it explicitly emphasizes anatomical interfaces and helps the model preserve structural transitions between regions. In the final experiment, the boundary kernel size was set to `boundary_k=3`.

The quantization module is implemented with `VectorQuantizer2`, using a codebook of learned embeddings and a corrected loss formulation with `legacy=False`. In this project, the codebook size was fixed to `n_embed=32` and the embedding dimension to `embed_dim=256`. The quantizer returns both the quantized latent tensor and the corresponding token indices, with a spatially meaningful index layout enabled by `sane_index_shape=True`. This makes it possible to treat the latent output as a 3D token grid rather than as a flattened sequence, which is particularly convenient for the subsequent prior.

4.2 3D positional encoding and latent tokenization

Before quantization, the deepest encoder representation is processed by a sinusoidal 3D positional encoding module. The implementation defines a `PositionalEncoding3D` block and a channel-first wrapper `PositionalEncodingPermute3D`, which inject sinusoidal position-dependent features along the three spatial axes. In the encoder, this positional signal is added to the bottleneck representation with a small scaling factor, according to the rule

$$x_5 \leftarrow x_5 + 0.1 \cdot \text{PE}_{3D}(x_5).$$

This means that the quantizer does not operate on purely appearance-free local descriptors, but on latent features that also contain explicit spatial context. Such positional information is relevant in anatomical modeling because the same semantic label may play different roles depending on its spatial location within the organ.

After positional encoding, the bottleneck tensor is quantized against the learned codebook. During training, a small Gaussian perturbation can also be added before quantization, which acts as a mild regularizer. The quantized latent tensor is then decoded back to full segmentation logits, while the token indices are retained as the discrete anatomical representation. In the final trained model, each preprocessed volume of size $(192, 192, 64)$ was compressed into a token grid of size $(12, 12, 4)$. Therefore, the tokenizer learns a very compact 3D symbolic representation of the anatomical mask, which is later used as the state space for the diffusion prior.

4.3 Loss function and evaluation metrics

The tokenizer is optimized as a segmentation-reconstruction model. The final implementation evaluates reconstruction quality using three complementary metrics: Dice score, Hausdorff distance (HD), and average surface distance (ASD). Dice measures regional overlap between predicted and target segmentations, while HD and ASD provide geometric information on the discrepancy between predicted and real surfaces. In the code, HD and ASD are computed with an automatic CPU/GPU fallback in order to avoid failures during surface metric evaluation.

4.4 DICE Coefficient (DICE)

The DICE coefficient measures the spatial overlap between the predicted mask and the actual segmentation:

$$\text{DICE} = \frac{2|A \cap B|}{|A| + |B|}.$$

It is the most widely used metric in medical image segmentation because it directly quantifies the proportion of the anatomical region that is correctly reconstructed. DICE is particularly appropriate for the prostate, where the Transition Zone (TZ) and Peripheral Zone (PZ) may have unbalanced sizes. However, DICE is sensitive to small structures: misclassifying even a few voxels in a small region can lead to a large decrease in the score. This sensitivity is essential for detecting segmentation failures that might not be visible solely through loss values.

4.5 Hausdorff Distance (HD)

The Hausdorff Distance evaluates the maximum surface discrepancy between predicted and actual boundaries. Formally, it calculates the greatest distance from any point on one surface to the nearest point on the other. Because the HD is strongly influenced by outliers, it highlights errors such as poorly segmented isolated voxels or jagged boundary peaks. Therefore, the HD provides a measure of robustness and maximum boundary error, which is crucial in clinical applications where localized boundary deviations can indicate anatomically implausible predictions.

4.6 Average Surface Distance (ASD)

The Average Surface Distance complements the HD by measuring the average distance between surfaces instead of the maximum deviation:

$$\text{ASD} = \frac{1}{|S_A|} \sum_{p \in S_A} d(p, S_B),$$

where $d(p, S_B)$ is the distance from point p on the prediction surface S_A to the reference surface S_B . ASD is less sensitive to outliers and captures the overall geometric similarity between

surfaces. In prostate segmentation, ASD is useful for detecting smooth and systematic contour changes that, while they may not significantly affect DICE or HD individually, do degrade the quality of the reconstruction.

4.7 Multimetric Score

Since DICE, HD, and ASD capture different aspects of segmentation performance, a weighted multi-objective score was used [2, 1]:

$$\text{multi} = w_D \cdot \text{DICE} + w_{HD} \cdot \frac{1}{1 + \text{HD}} + w_{ASD} \cdot \frac{1}{1 + \text{ASD}}.$$

This formulation ensures that lower HD and ASD values contribute positively to the final score, keeping all terms on comparable scales. The multimetric criterion is crucial when selecting control points, as it avoids the risk of choosing a model with good performance in terms of region overlap (DICE) but with low geometric accuracy (HD or ASD). In the context of generating tokenized anatomical representations for diffusion models [1], it is especially important to prioritize segmentations with smooth and anatomically consistent boundaries.

4.8 Tokenizer objective and metrics

The tokenizer is trained as a segmentation-reconstruction model. The loss has two parts: a segmentation loss and an embedding loss. In simplified form,

$$L = L_{seg} + w_{emb}L_{emb}.$$

The segmentation term is a Dice + cross-entropy loss. The embedding term comes from vector quantization. During training, the weight of the embedding term is increased with a warm-up schedule.

Validation and test performance are measured with Dice, Hausdorff distance (HD), and average surface distance (ASD). A composite score is also used for checkpoint selection (Multimetric Score) that gives more weight to overlap quality but still includes surface quality.

For the final experiment, the tokenizer was trained in a three-class setting (background, TZ, and PZ) with `channels=16`, `embed_dim=256`, and a codebook size of `n_embed=32`. The model was trained for 50 epochs. During training, the script monitors not only the validation Dice score but also Hausdorff distance (HD), average surface distance (ASD), and a composite score denoted `val_multi`. Two best checkpoints are therefore maintained: one selected by Dice and one selected by the composite metric. In the experiment reported here, both selection criteria converged to the same checkpoint at epoch 49, indicating that the best reconstruction model was also the best according to the multi-metric criterion.

4.9 Perplexity and embedding usage during training

The training script explicitly monitors codebook statistics through the function `codebook_metrics`, which computes the fraction of used embeddings and the token perplexity from the latent histogram. During training, these values are logged epoch by epoch, and representative batch-level summaries are also printed, including the most frequent token ids in the current latent batch.

The early training dynamics showed clear signs of token concentration. For example, at first epochs, representative batches exhibited usage fractions as low as 0.155–0.250 and perplexity values around 1.9–3, indicating that only a very small number of embeddings carried most of the latent mass.

As training progressed, the codebook usage became richer, although still limited. By the final epochs, representative batches reached usage fractions in the range 0.38–0.41 and perplexity values around 9.0–1.0. This shows that the latent representation diversified over time and

that more embeddings became involved in the encoding process. However, the effective usage remained well below the nominal codebook size of 32. Therefore, the final tokenizer should be interpreted as having learned a *structured but partially collapsed* discrete latent space: it clearly organizes anatomy into consistent symbolic patterns, but it does not fully exploit the theoretical capacity of the embedding dictionary.

4.10 Training curves and checkpoint selection

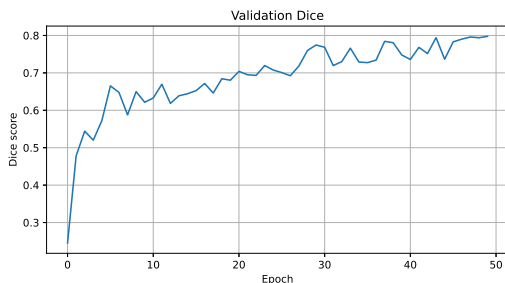
The VQ-VAE was trained for 50 epochs. According to the Lightning logs, the training became clearly stable in the second half of optimization. The validation Dice, validation multi-score, and validation loss all improved over time, while the codebook-usage and perplexity curves also became more stable. According to Table 1, the best checkpoint was obtained at epoch 49 for both model-selection criteria. The monitored values were:

$$\text{val_dice} = 0.7975, \quad \text{val_multi} = 0.6812. \quad (1)$$

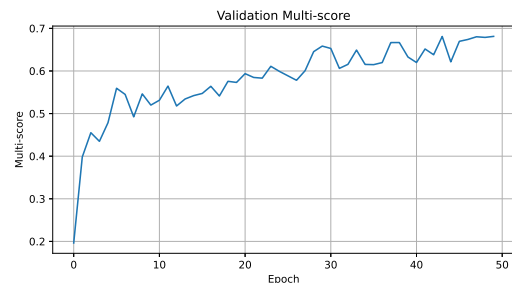
This means that the same late-epoch model was preferred both by overlap quality and by the combined overlap-and-surface criterion. According to Figure 1, the main validation metrics evolve in a consistent way, and Figure 2 shows that codebook usage and perplexity also become more stable near the end of training.

Table 1: Tokenizer checkpoint selection after 50 epochs.

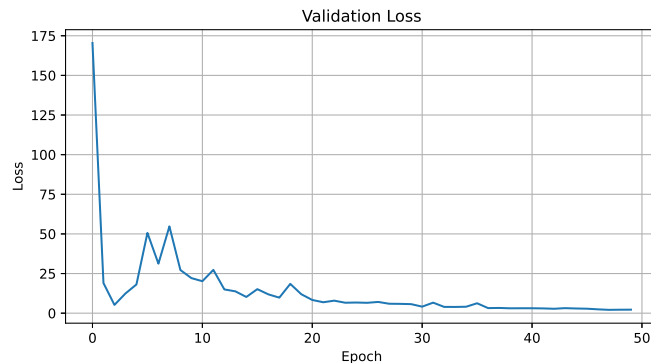
Criterion	Best epoch	Validation Dice	Validation multi-score
Best Dice checkpoint	49	0.7975	0.6812
Best Multi checkpoint	49	0.7975	0.6812



(a) Validation Dice over 50 epochs.



(b) Validation multi-score over 50 epochs.



(c) Validation loss over 50 epochs.

Figure 1: Main validation curves of the VQ-VAE training.

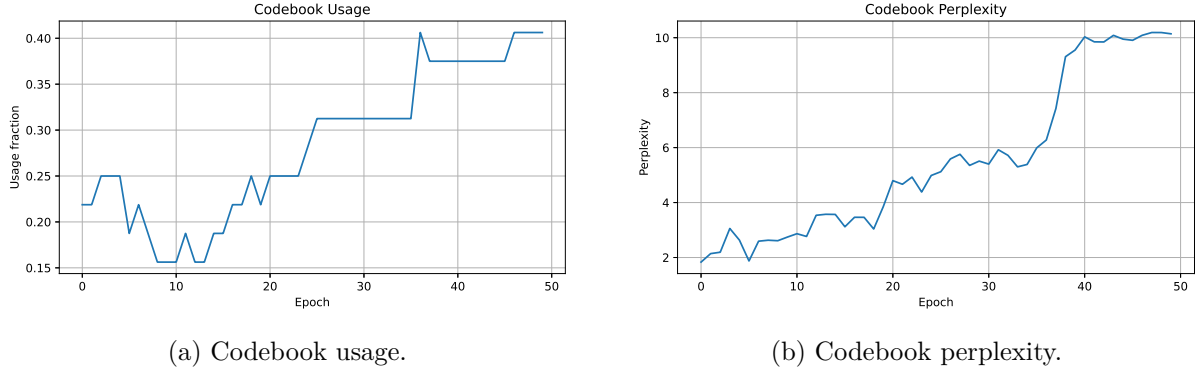


Figure 2: Codebook-related curves during VQ-VAE training.

4.11 Test performance on the 12 cases

The final tokenizer was evaluated on the 12 test masks. Table 2 reports the metric values for each test case. The mean values are:

$$\text{Dice} = 0.8025, \quad \text{HD} = 11.9389, \quad \text{ASD} = 1.6778. \quad (2)$$

These results show that the tokenizer reconstructs the prostate structures with good overlap and moderate surface error. The Dice values are stable across the 12 cases. HD varies more strongly, which is expected because HD is sensitive to local contour deviations.

Table 2: Test results of the trained VQ-VAE on the 12 test samples.

Test sample	Dice	HD	ASD
1	0.8043	8.1777	1.6139
2	0.8446	21.6391	1.2047
3	0.8362	10.1178	1.3508
4	0.7782	10.7527	1.7404
5	0.7401	11.6374	2.2040
6	0.8345	14.9215	2.1496
7	0.7490	7.9681	2.2705
8	0.8013	9.6060	1.2609
9	0.8191	12.5400	1.4421
10	0.7993	13.0476	1.7037
11	0.8130	13.6086	1.3614
12	0.8106	9.2496	1.8317
Mean	0.8025	11.9389	1.6778
Std.	0.0327	3.7510	0.3746

5 Discrete Latent Space and Token Analysis

A central objective of the first stage of the pipeline is not only to reconstruct anatomical masks accurately, but also to learn a discrete latent representation that is compact, stable, and informative enough to support downstream generative modeling. For this reason, the VQ tokenizer was analyzed beyond reconstruction metrics, with special attention to token export, codebook coverage, token usage dynamics, and the spatial structure of the learned discrete representations.

5.1 Export of latent token grids

After training, the script `trainprostVQ.py` exports the token indices obtained from the encoder-quantizer stage for the training, validation, and test subsets. This is done by encoding each preprocessed segmentation mask through the trained tokenizer and saving the resulting discrete indices as `tokens_train.pt`, `tokens_val.pt`, and `tokens_test.pt`. The export is performed without adding encoder noise, so the saved tensors correspond to the deterministic tokenized anatomical representation learned by the model.

According to Table 3, the exported tensors had the following shapes:

`tokens_train` : (55, 12, 12, 4), `tokens_val` : (12, 12, 12, 4), `tokens_test` : (12, 12, 12, 4).

These dimensions confirm that each full-resolution prostate mask was compressed into a compact latent grid of size $12 \times 12 \times 4$. This compression factor is substantial, since the original standardized volumes have size (192, 192, 64), and it defines the latent domain on which the discrete prior is later trained.

5.2 Codebook coverage and effective latent capacity

Although the tokenizer was configured with a codebook size of `n_embed=32`, the exported token tensors revealed that only 10 token ids were active across the real train, validation, and test sets. The total number of unique active codes was consistently equal to 10. This corresponds to an effective codebook utilization of

$$\frac{10}{32} = 31.25\%.$$

From the perspective of representation learning, this result is mixed. On the positive side, the fact that train, validation, and test all exhibited the same number of active tokens suggests that the learned discrete representation was stable across data splits and did not suffer from strong split-dependent drift. On the other hand, the tokenizer did not make full use of the available latent vocabulary. In other words, the codebook was large enough to offer additional representational capacity, but the model converged to a more restricted subset of embeddings.

This partial codebook underuse is important because it already anticipates one of the limitations observed in the generative stage. If the tokenizer itself concentrates anatomical structure into a relatively small subset of codes, the prior trained on top of this representation is likely to inherit a similarly concentrated latent distribution.

5.3 Dominant token identities and class-dependent behavior

The logs show that the background class occupied the vast majority of latent positions throughout training, which is expected given the spatial sparsity of the prostate zones relative to the full padded volume. Across epochs, a small set of token ids became consistently dominant, especially 31, 23, 18, 13 and 4. These same ids are also those that dominate the exported histograms, confirming that the latent organization is not random but highly structured.

5.4 Visualization of latent slices and token histograms

The script `tokens.py` was used to inspect the exported token tensors. For each split, it prints the tensor type, shape, data type, value range, and number of unique active tokens, and it also saves two visual diagnostics: a central latent slice from the first sample and a histogram of token frequencies.

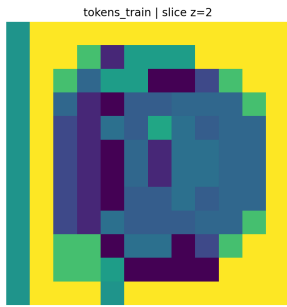
According to Figures 3, 4, and 5, the resulting histograms are highly consistent across train, validation, and test. Rather than a uniform distribution over the 32 available codes, they show a concentrated pattern in which a few codes dominate the latent space. In particular, token

ids 31, 23, 18, 13 and 4 account for most of the latent occupancy, while the remaining active codes appear with much lower frequencies. This confirms that the tokenizer learned a stable discrete vocabulary but also reinforces the conclusion that the latent space remains relatively concentrated.

The slice visualizations provide complementary information. They show that the token grids are not spatially random, but instead exhibit coherent regional organization. Neighboring latent positions often share the same token id over extended areas, which suggests that the tokenizer captures anatomical continuity and coarse shape structure in a spatially meaningful way. This is important for the next stage of the pipeline, because the discrete prior does not generate isolated tokens independently; it must learn to generate full 3D token grids with structured neighborhood relations.

Table 3: Token-space summary for the real data splits.

Split	Tensor shape	Token range	Unique active tokens
Train	(55, 12, 12, 4)	[4, 30]	10
Validation	(12, 12, 12, 4)	[4, 30]	10
Test	(12, 12, 12, 4)	[4, 30]	10

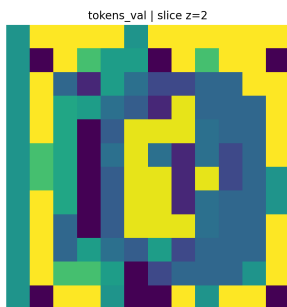


(a) Training token slice.

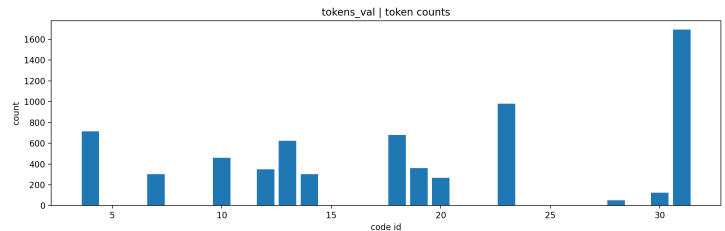


(b) Training token histogram.

Figure 3: Token diagnostics for the training split.



(a) Validation token slice.



(b) Validation token histogram.

Figure 4: Token diagnostics for the validation split.

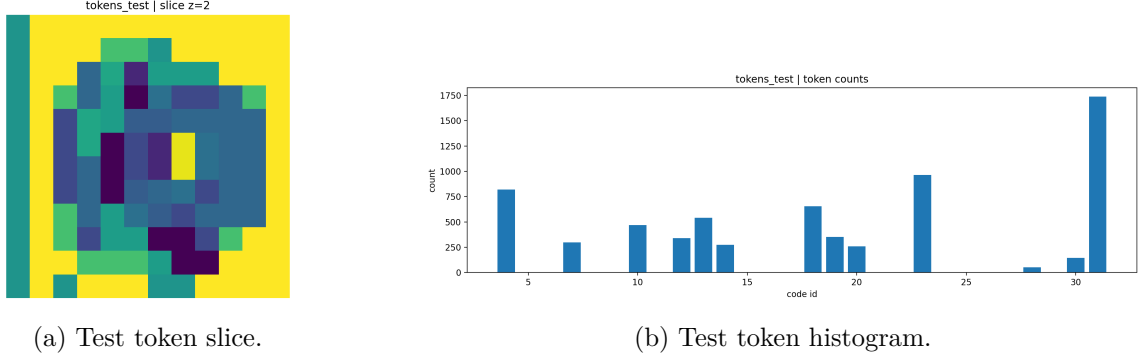


Figure 5: Token diagnostics for the test split.

The histograms show a small group of dominant token ids. This result is important for the second stage. It means that the diffusion prior does not need to model all 32 tokens equally often, but it also means that the latent space may limit diversity.

6 Discrete Diffusion Prior over Token Grids

Once the anatomical tokenizer has learned a compact discrete representation of prostate shapes, the second stage of the pipeline consists of learning a generative prior directly in that latent space. Importantly, this stage no longer models full-resolution segmentation masks in voxel space. Instead, it operates on discrete token grids of size $12 \times 12 \times 4$, previously exported by the tokenizer. The objective is therefore to learn a distribution over latent anatomical codes and to generate new token volumes that can later be decoded back into full 3D segmentations.

The second stage learns a prior directly over the discrete token grids. Let x_0 be a clean token volume. During the forward corruption process, a subset of token positions is replaced by random token ids according to a cosine schedule. The denoiser then receives a noisy grid x_t and the time step t , and predicts logits over the token vocabulary for each spatial location.

The denoiser uses token embeddings, time embeddings, 3D convolutions, and residual blocks. The baseline model uses 200 diffusion steps, hidden dimension 256, and 6 residual blocks. Several variants were tested by changing the number of diffusion steps, the hidden size, the number of blocks, the cosine schedule parameter s , and the sampling temperatures.

6.1 Denoiser architecture

The reverse model used to predict clean tokens from noisy ones is `TokenDenoiser3D`, implemented in `token_diffusion.py`. It is a fully convolutional 3D denoiser operating directly on discrete token volumes. The architecture consists of four main components:

- a token embedding layer that maps each discrete code id to a continuous feature vector of dimension d_{model} ;
- a time embedding layer that maps the diffusion step t to another vector in the same feature space;
- an input $3 \times 3 \times 3$ convolution followed by a stack of residual 3D convolutional blocks;
- a final $1 \times 1 \times 1$ convolution that outputs logits over the K possible token identities at each latent position.

More precisely, if x_t is the corrupted token volume, the model first embeds each token id, permutes the embedding dimension to the channel axis, adds the broadcasted time embedding,

and then applies a sequence of residual 3D blocks. The final output has shape

$$[B, K, H, W, D],$$

that is, one logit vector per spatial location and per batch element.

In the present implementation, the denoiser was configured with

$$d_{\text{model}} = 256, \quad n_{\text{blocks}} = 6.$$

This design is intentionally simple: instead of using a full U-Net at the latent level, the prior relies on a compact residual 3D network, which is sufficient given the small spatial size of the token grid.

6.2 Experimental design

The discrete prior was not tested only once. According to Table 4, a full comparison was done over nine configurations: one baseline experiment and eight variants. The experiments changed one of the following factors: number of diffusion steps, denoiser size, denoiser depth, cosine schedule parameter (s), or sampling temperatures. This comparison is important because the best denoising loss does not necessarily give the best decoded anatomical samples.

Table 4: Experimental settings for all token-prior experiments.

Experiment	num_steps	d_model	n_blocks	s	$temp_{hi}/temp_{lo}/temp_{split}$
Exp. 0	200	256	6	0.008	1.25 / 1.05 / 0.60
Exp. 1.1	100	256	6	0.008	1.25 / 1.05 / 0.60
Exp. 1.2	400	256	6	0.008	1.25 / 1.05 / 0.60
Exp. 2.1	200	128	4	0.008	1.25 / 1.05 / 0.60
Exp. 2.2	200	384	8	0.008	1.25 / 1.05 / 0.60
Exp. 3.1	200	256	6	0.001	1.25 / 1.05 / 0.60
Exp. 3.2	200	256	6	0.020	1.25 / 1.05 / 0.60
Exp. 4.1	200	256	6	0.008	1.10 / 1.00 / 0.60
Exp. 4.2	200	256	6	0.008	1.50 / 1.20 / 0.70

6.3 Comparison of the nine diffusion experiments

Table 5 summarizes the main results. Experiment 0 gives the best validation loss, with 0.1852 at epoch 48. However, Experiment 1.1 gives the best decoded-sample similarity metrics, including the best foreground matching score and the best foreground Dice score. Therefore, the comparison suggests that the lowest denoising loss is not always the best criterion for final anatomical quality after decoding.

Table 5: Comparison of all token-prior experiments. Bold indicates the best value when the metric has a clear direction.

Experiment	Best val_loss	Best ep.	GEN uniq.	GEN bg	match_fg	dice_fg	Near mean / max
Exp. 0	0.1852	48	13	0.2977	0.6676	0.8353	0.7179 / 0.7917
Exp. 1.1	0.4896	26	14	0.2413	0.8121	0.9281	0.4850 / 0.5278
Exp. 1.2	0.5874	10	14	0.2378	0.5914	0.8442	0.6517 / 0.6823
Exp. 2.1	0.5001	6	13	0.2413	0.7399	0.8805	0.5836 / 0.5972
Exp. 2.2	0.4630	11	13	0.2613	0.5469	0.7395	0.5063 / 0.5278
Exp. 3.1	0.4555	15	13	0.2431	0.7106	0.8719	0.5167 / 0.5538
Exp. 3.2	0.4285	19	14	0.4071	0.6780	0.8372	0.5922 / 0.6458
Exp. 4.1	0.4497	12	12	0.3316	0.7004	0.8813	0.6621 / 0.7014
Exp. 4.2	0.5531	9	13	0.1033	0.6129	0.8066	0.5382 / 0.5764

The columns of this table summarize both training performance and generation behavior for each diffusion experiment. **Experiment** identifies the tested setting. **Best val_loss** is the lowest validation loss reached during training, and **Best ep.** is the epoch where this value was obtained. **GEN uniq.** shows how many different codebook tokens appear in the generated samples, which indicates token diversity. **GEN bg** is the relative frequency of the background token in the generated samples, where the background token is defined as the most frequent token in the training set. **match_fg** measures the proportion of equal voxel labels between two generated samples only inside the foreground region, while **dice_fg** gives the binary Dice score between their foreground masks. Finally, **Near mean / max** reports the mean and maximum similarity between each generated token grid and its nearest training token grid, and it is used as a simple memorization check.

A few trends are clear. First, the generated token vocabulary remains smaller than the full codebook in all experiments. The generated token count stays between 12 and 14 active tokens. Second, some variants improve decoded anatomical similarity even when their validation loss is worse than the baseline. Third, Experiment 1.1 offers the best compromise for visual quality and decoded similarity, which is why it is used for the final sample grid in this report.

6.4 Interpretation of the diffusion results

The diffusion model learns meaningful token patterns. The real/reconstruction/generated comparisons show that the prior produces plausible prostate shapes after decoding. The generated shapes keep the global form of the gland and the main zone organization. At the same time, the experiment comparison also shows that the model family still has limited diversity. This is consistent with the VQ-VAE token analysis: only a subset of tokens is heavily used on real data, so the prior learns a concentrated latent distribution.

7 Visual comparison of the nine diffusion experiments

This section groups the qualitative figures that compare one real mask, its VQ reconstruction, a generated sample, and a mismatch view. According to Figures 6, 7, 8, 9, 10, 11, 12, 13, and 14, these comparisons are important because they show the practical effect of each experiment after decoding.

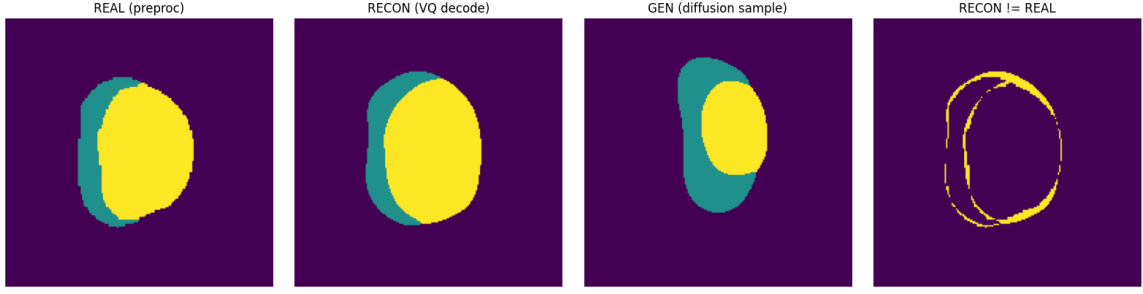


Figure 6: Experiment 0 (baseline): real mask, VQ reconstruction, generated sample, and mismatch view.

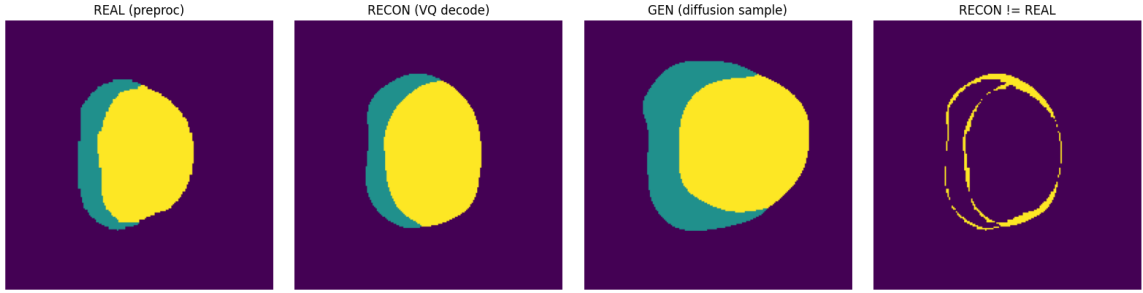


Figure 7: Experiment 1.1 with 100 diffusion steps: real mask, VQ reconstruction, generated sample, and mismatch view.

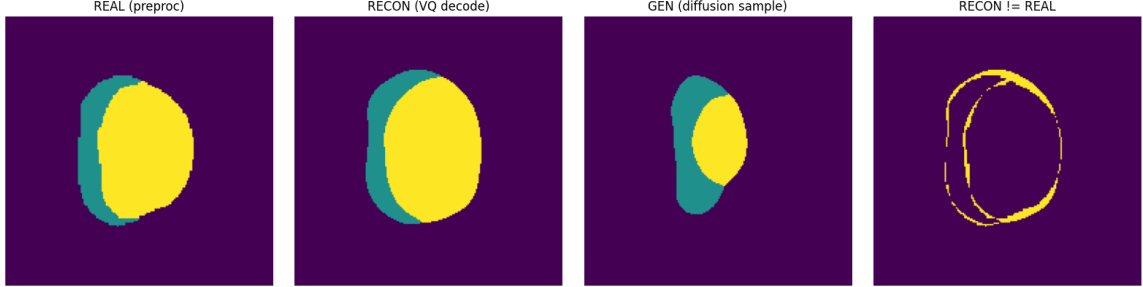


Figure 8: Experiment 1.2 with 400 diffusion steps: real mask, VQ reconstruction, generated sample, and mismatch view.

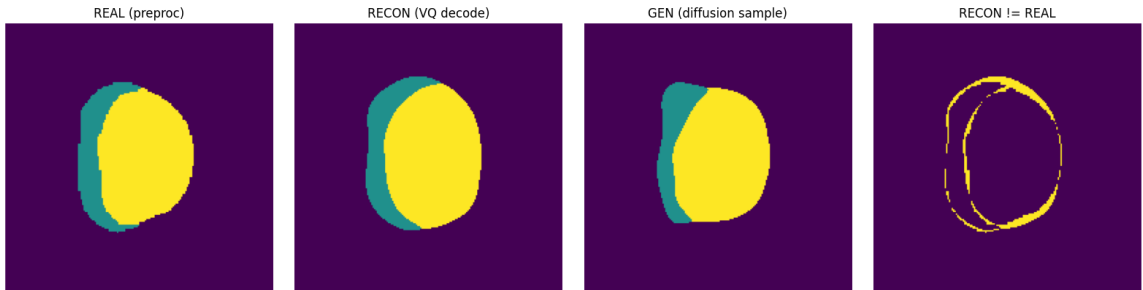


Figure 9: Experiment 2.1 with a smaller denoiser: real mask, VQ reconstruction, generated sample, and mismatch view.

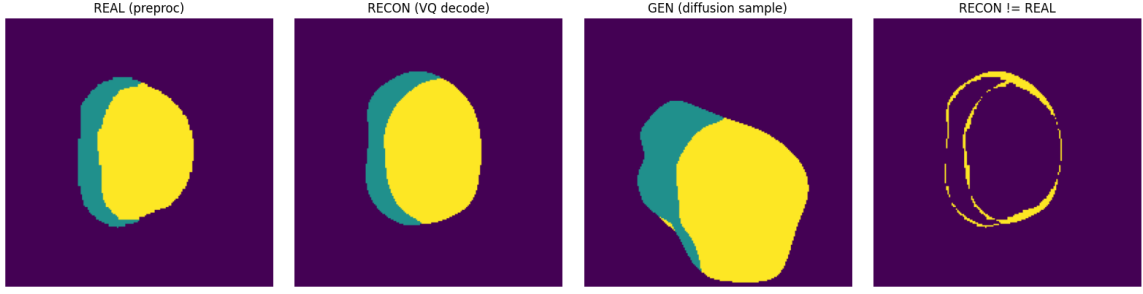


Figure 10: Experiment 2.2 with a larger denoiser: real mask, VQ reconstruction, generated sample, and mismatch view.

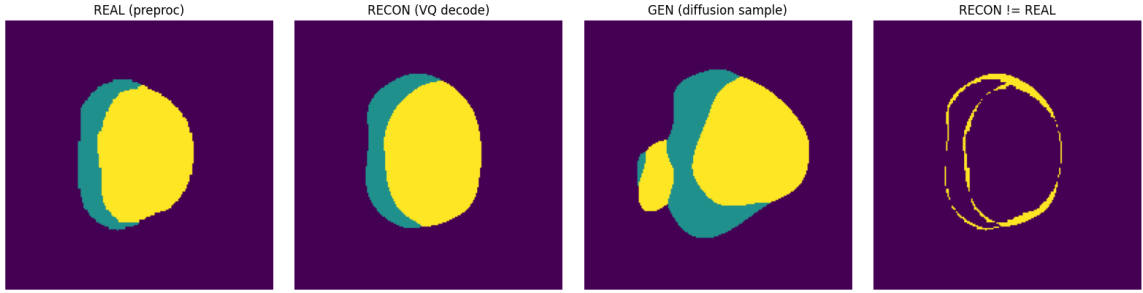


Figure 11: Experiment 3.1 with a smaller cosine-schedule offset: real mask, VQ reconstruction, generated sample, and mismatch view.

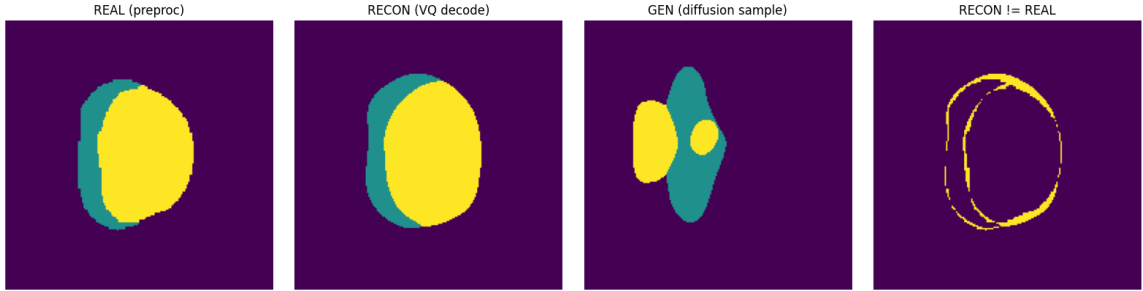


Figure 12: Experiment 3.2 with a larger cosine-schedule offset: real mask, VQ reconstruction, generated sample, and mismatch view.

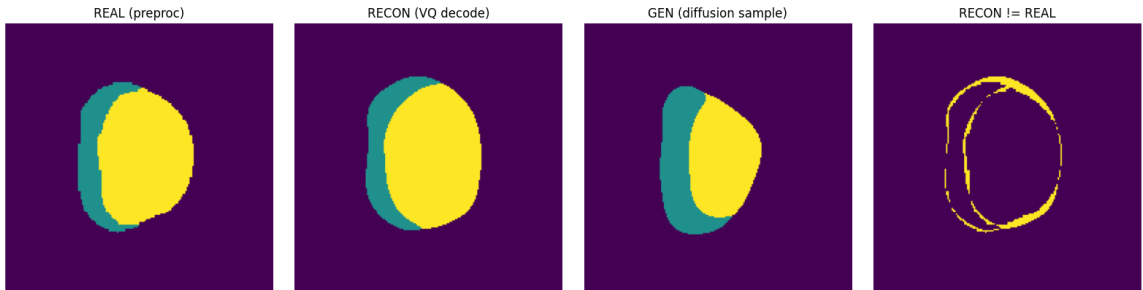


Figure 13: Experiment 4.1 with lower sampling temperatures: real mask, VQ reconstruction, generated sample, and mismatch view.

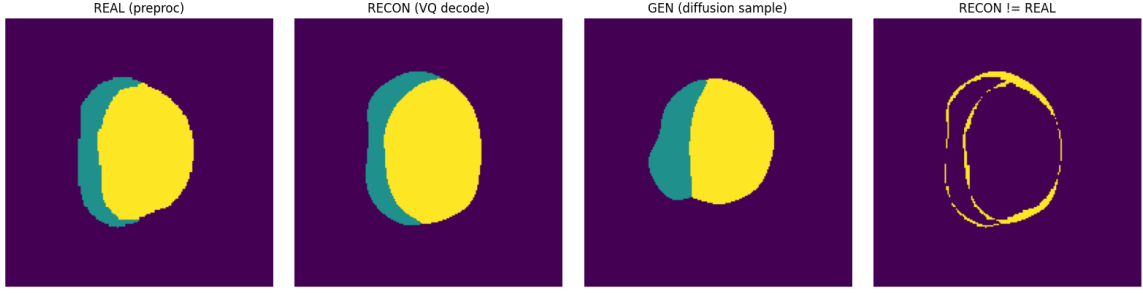


Figure 14: Experiment 4.2 with higher sampling temperatures: real mask, VQ reconstruction, generated sample, and mismatch view.

8 Final decoded generated samples

Figure 15 shows the eight final decoded samples selected for presentation. These volumes are anatomically plausible. They keep the main prostate shape and the internal zone arrangement. However, the samples are still visually close to each other, which supports the conclusion that diversity remains limited. This visual result is in agreement with the metrics of Experiment 1.1 in Table 5 , which suggest that the generated samples are coherent but not highly diverse.

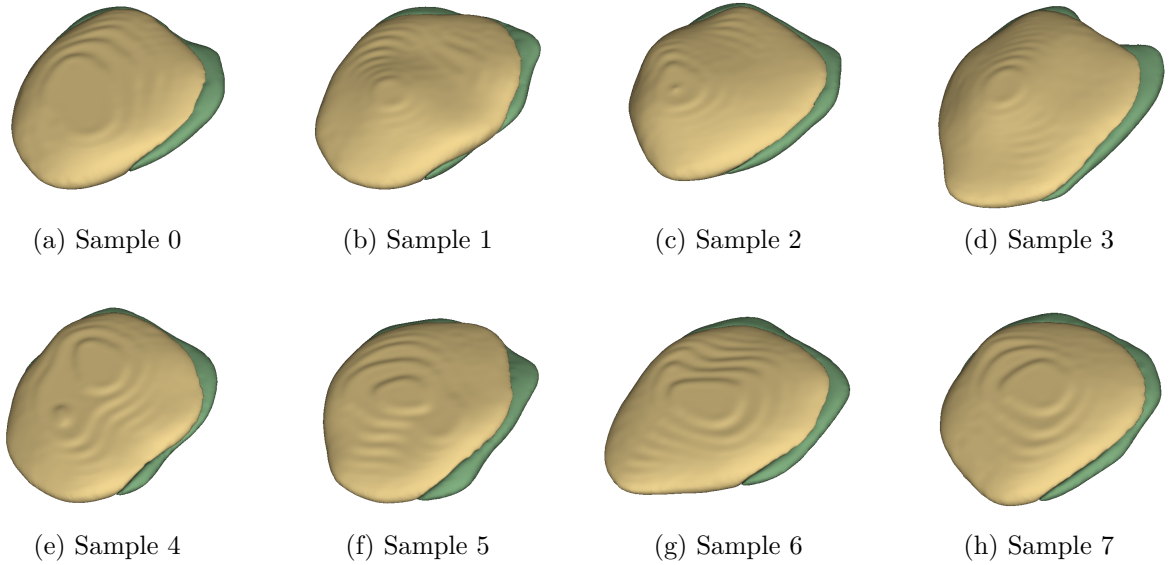


Figure 15: Final decoded generated samples.

9 Discussion

The results support three main conclusions. First, the VQ-VAE tokenizer learned a useful compact representation of prostate anatomy. According to Table 2, the reconstruction scores on the test set are good enough to support the next stage. Second, the token space is stable across train, validation, and test splits, but according to Table 3 and Figures 3–5, it uses only a limited fraction of the codebook. Third, the discrete diffusion prior can generate plausible token grids and plausible decoded masks, but the current model family still concentrates around a narrow region of the latent space.

The comparison between the nine diffusion experiments is also informative. According to Table 5, the best training loss is not the only useful criterion. If the final goal is anatomical generation after decoding, then decoded-sample quality must also be considered. In this report,

Experiment 0 is the best by validation loss, but Experiment 1.1 is the best by decoded foreground similarity and is therefore the most convincing visual result. The qualitative figures support the same conclusion: the decoded samples preserve the global prostate structure and the internal zone arrangement, but they remain visually close to each other. This suggests that the current bottleneck is not anatomical plausibility, but diversity. In practical terms, the tokenizer already captures stable structure, while the prior still explores a limited part of the learned latent space.

10 Conclusion

This work implemented a full discrete anatomical generation pipeline for prostate masks. The VQ-VAE stage, adapted from the public implementation of AinkaranSanthi [5], compressed the anatomy into a compact token grid of size (12, 12, 4) and reached its best checkpoint at epoch 49 after 50 training epochs. According to Table 2, the test reconstruction results were strong, with mean Dice 0.8025, mean HD 11.9389, and mean ASD 1.6778. The token analysis showed stable but partial codebook usage, with 10 active tokens out of 32.

The discrete diffusion stage compared nine configurations. According to Table 5, the baseline gave the best validation loss, while Experiment 1.1 gave the best decoded anatomical similarity. The final decoded samples in Figure 15 are coherent and anatomically plausible, which confirms that the two-stage discrete pipeline works in the prostate domain. At the same time, the limited number of active codes and the visual similarity between generated samples show that diversity is still the main weakness of the current system. The next step is therefore not only to generate plausible masks, but to enlarge the explored latent space without losing anatomical consistency. Possible directions include a richer token space, larger training data, stronger regularization against token concentration, or a more expressive token prior.

References

- [1] Minghui Hu et al. “Global Context with Discrete Diffusion in Vector Quantised Modelling for Image Generation”. In: *CoRR* abs/2112.01799 (2021). arXiv: 2112.01799. URL: <https://arxiv.org/abs/2112.01799>.
- [2] Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. “Neural Discrete Representation Learning”. In: *CoRR* abs/1711.00937 (2017). arXiv: 1711.00937. URL: <http://arxiv.org/abs/1711.00937>.
- [3] B Nicolas Bloch, Ashali Jain, and C. Carl Jaffe. *Data From PROSTATE-DIAGNOSIS*. 2015. DOI: 10.7937/K9/TCIA.2015.FOQEUJVT. URL: <https://www.cancerimagingarchive.net/collection/prostate-diagnosis/>.
- [4] Geert Litjens, Jurgen Futterer, and Henkjan Huisman. *Data From Prostate-3T*. 2015. DOI: 10.7937/K9/TCIA.2015.QJTV5IL5. URL: <https://www.cancerimagingarchive.net/collection/prostate-3t/>.
- [5] AinkaranSanthi. *Vector-Quantisation-for-Robust-Segmentation*. <https://github.com/AinkaranSanthi/Vector-Quantisation-for-Robust-Segmentation>. June 2022. URL: <https://github.com/AinkaranSanthi/Vector-Quantisation-for-Robust-Segmentation>.
- [6] B. Nicholas Bloch et al. *NCI-ISBI 2013 Challenge: Automated Segmentation of Prostate Structures (ISBI-MR-Prostate-2013)*. 2015. DOI: 10.7937/K9/TCIA.2015.ZFOVL0PV. URL: <https://www.cancerimagingarchive.net/analysis-result/isbi-mr-prostate-2013/>.